Cubic Gradient-Based Material Interfaces

Iuri Prilepov, Harald Obermaier, *Member*, *IEEE Computer Society*, Eduard Deines, Christoph Garth, *Member*, *IEEE*, and Kenneth I. Joy, *Member*, *IEEE*

Abstract—Multifluid simulations often create volume fraction data, representing fluid volumes per region or cell of a fluid data set. Accurate and visually realistic extraction of fluid boundaries is a challenging and essential task for efficient analysis of multifluid data. In this work, we present a new material interface reconstruction method for such volume fraction data. Within each cell of the data set, our method utilizes a gradient field approximation based on trilinearly blended Coons-patches to generate a volume fraction function, representing the change in volume fractions over the cells. A continuously varying isovalue field is applied to this function to produce a smooth interface that preserves the given volume fractions well. Further, the method allows user-controlled balance between volume accuracy and physical plausibility of the interface. The method works on two- and three-dimensional Cartesian grids, and handles multiple materials. Calculations are performed locally and utilize only the one-ring of cells surrounding a given cell, allowing visualizations of the material interfaces to be easily generated on a GPU or in a large-scale distributed parallel environment. Our results demonstrate the robustness, accuracy, and flexibility of the developed algorithms.

Index Terms—Visualization, boundary representations, computational geometry and object modeling

1 Introduction

Interface reconstruction and tracking is an important problem with broad application in combustion, climate studies, and astrophysics simulations. The boundaries between sloshing fluids in tanks, between cavity and casting in mold filling applications, between fluids and embedded objects, and between organs in the human body all fit within this research area. Multifluid hydrodynamics simulation codes, for example, often produce data sets in which cell-centered scalar quantities report the fractional volume of each material contained within the cell. The challenge is to use these *volume fractions* to reconstruct the interfaces between materials, generating smooth, continuous boundary interfaces between regions of homogeneous materials.

One of the primary difficulties in reconstructing material interfaces from volume fraction data is that the problem is underconstrained, because there are an infinite number of boundary interfaces that can satisfy a given set of volume fractions.

Some of the most common materials represented in volume fraction data are fluids. Therefore, besides providing an accurate reconstruction of volume fractions, the goal is to develop a material interface reconstruction algorithm that produces a reasonably smooth, fluid-like appearance and, therefore, physically more realistic result. This will be the main focus for our paper.

Ideally, the reconstructed material interfaces should be continuous across cell boundaries and segment cells into regions that closely approximate the given volume fractions. A corresponding algorithm should be fast, have a small memory footprint, and be parallelizable, both on GPUs and on large scale distributed parallel systems.

We present a new method to visualize material interfaces that is based upon a volume-accurate per-cell approximation of the material interface. The method generates two continuous functions over the domain: A continuous volume fraction function that matches given gradients at the vertices defining a cell and a continuously varying isovalue function that is used to produce the final interface. Manipulation of the isovalue function allows for control over the accuracy and the visual representation of the interface. Gradient-based approximation of the volume fraction function can naturally represent many interesting interface topologies, which cannot be reproduced with existing techniques comparable in performance.

The proposed algorithm generalizes existing, widely accepted interface reconstruction methods. It allows for the control over reconstruction accuracy and quality of the interface. Since all our calculations are highly local, all stages of the algorithm can be easily parallelized to achieve good performance.

Specifically, the primary contributions of this paper are as follows:

- A new volume accurate per-cell approximation to the material interface is defined in Section 3. This approximation is based upon a continuous volume fraction function that matches gradients defined at the vertices of the cells. It allows multimaterial cells to be addressed, and allows material interfaces with greater detail to be generated.
- A new method to generate a material interface, with control over the balance between carefully matching

- *I. Prilepov is with the Computer Science Department, Institute for Data Analysis and Visualization, UC Davis, 1 Shields Ave, Academic Surge 2077, Davis, CA 95616. E-mail: yprilepov@gmail.com.*
- *H. Obermaier is with the Computer Science Department, Institute for Data Analysis and Visualization, UC Davis, 1 Shields Ave, Academic Surge 2075, Davis, CA 95616.*
- *K.I. Joy is with the Computer Science Department, Institute for Data Analysis and Visualization, UC Davis, 1 Shields Ave, Academic Surge 2079, Davis, CA 95616.*
- *E. Deines is with CD-adapco, 10800 Pecan Park Blvd, Suite 210, Austin, TX 78750.*
- *C. Garth is with Technische Universität Kaiserslautern, Postfach 3049, D-67653 Kaiserslautern, Germany.*

Manuscript received 17 June 2012; revised 29 Nov. 2012; accepted 23 Jan. 2013; published online 13 Feb. 2013.
Recommended for acceptance by P. Cignoni.
For information on obtaining reprints of this article, please send e-mail to: tvcg@computer.org, and reference IEEECS Log Number TVCG-2012-06-0111.
Digital Object Identifier no. 10.1109/TVCG.2013.16.

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.
1077-2626/13/$31.00 © 2013 IEEE Published by the IEEE Computer Society

input volume fractions and producing a smooth interface more appropriate for the analysis of fluid dynamics. This functionality is explained in Section 4.
- A method that is naturally suitable for parallel implementation, either on large scale parallel systems or GPUs. Section 5 describes the implementation used in the paper.
- A technique that generalizes and improves upon widely accepted techniques. The results section showcases the material interfaces produced with our method.

2 Related Work

The reconstruction of interfaces from segmented grid data has been a staple problem in the visualization community for many years; the problem, simply stated, is to partition a labeled mesh into regions with homogeneous material labels. Binary labelings can be dealt with algorithms based on Marching Cubes [1], while multilabel data can be handled with additional rules and extended lookup tables, such as in work by Hege et al. [2] and several other researchers [3], [4], [5].

The problem we address in this paper, on the other hand, is the reconstruction of surfaces based upon volumetric constraints. A number of researchers have developed algorithms to address this problem. Youngs' method [6] and others [7], [8] attempt to find an "average gradient" that can be used as a normal to the material interface. This enables them to utilize a simple procedure to clip the cell such that the volume of the clipped portion is equal to the given volume fraction by placing a plane with the calculated normal appropriately within the cell. The method preserves volume fractions well, is very efficient, but is not continuous, and poorly approximates the surface in areas of large curvature or small detail.

Bonnell et al. [9], [10] move the problem to a dual grid, whose nodes are at the centers of the cells. This enables them to generate an algorithm that assumes that the data are given at the vertices of a grid. They generate the interfaces by considering points in a "barycentric space," calculating intersections in this space, and mapping them back to the dual grid. They require an a priori choice of isocontour in barycentric space (e.g., where the material value is $(\frac{1}{2}, \frac{1}{2})$), which is a rough approximation to the actual interface and does not preserve volume fractions. They utilize an algorithm similar to marching tetrahedra [11] to define the interface, and use a generalized clipping algorithm to handle the multimaterial case.

Meredith and Childs [12] average the cell-based volume fractions to the vertices, avoiding the use of a dual grid. They assume cell edge intersections occur at a point, where the linearly interpolated volume fraction values from the dual grid nodes are equal and are able to use fast table lookups to generate geometry, similar to Marching Cubes [1]. By comparing the volume fractions generated by their method against the input volume fractions, they can set up an iterative procedure to improve their result. The assumption of having a single intersection per edge limits the topologies produced by the algorithm. The adaptive procedure is not guaranteed to converge and resulting error in practice is still high.

Anderson et al. [13] take a discrete approach to the development of the interface. Their method is based upon optimizing the labeling of fractional volume elements in a discretization of the mesh. They initially label a discretization of each cell, and introduce a volume conservative swap move for optimization. Their technique has significant advantages over previous methods as it produces interfaces between multiple materials that are continuous across cell boundaries for time varying and static data in arbitrary dimension with bounded error. Unfortunately, the algorithm can take a long time to converge (on the order of several hours) because it requires a Pott's-Model-like calculation [14]. The voxelized output from this method can be processed as described in the work by Feng et al. [15] to obtain a smooth interface.

Refining the previous method, Anderson et al. [16] have taken a different approach by focusing on the interface itself, and not on the per-cell approximations. They utilize an adaptive interface method [17] on a piecewise linear approximation of the interface, iterating over the interface until the desired metrics of accuracy and smoothness are reached. However, the iterative procedure may fail to converge quickly and the volume error can still be high. Another difficulty is generating a first approximation to the interface, where they use a discrete approach similar to [13].

Several researchers [18], [19], [20] have developed algorithms specifically tailored toward fluid flow simulations. They normally assume that more than just volume fraction information in a cell is provided—volume fractions, area fractions on the faces, and edge fractions on edges, or data augmented with material centroid information.

3 Material Interface Approximation

We develop a method to create high-quality material interface reconstructions—i.e., boundary interfaces between different material regions—for meshes that contain cell-centered volume fraction data. We assume that volume fraction data exist within a spatial domain that has been decomposed into a finite grid of regular cells C. In an m-material setting, each cell $c \in C$ has an associated tuple $v_c = (v_1, \ldots, v_m)$, where the value v_i is the fractional volume of material i within the cell. Volume fractions are nonnegative ($v_i \geq 0$), and form a partition of unity over the volume of the cell ($\sum_{i=1}^{m} v_i = 1$). In general, a mesh will have *mixed cells*, cells that have nontrivial volume fractions, and *pure cells*, cells that contain only a single material. It is within the mixed cells where the boundaries exist. Fig. 1 illustrates the general problem of material interface reconstruction.

We will first discuss the two-material case over a two-dimensional mesh, and then generalize the method to multimaterial problems defined over three-dimensional meshes. We will assume that a mesh M contains a set of regular cells $C_{i,j}$ arranged in a two-dimensional grid, each having an edge length of 1. We assume a standard parameterization over the mesh where $(x, y) \in C_{i,j}$ if and only if $i \leq x \leq i+1$ and $j \leq y \leq j+1$. Each $C_{i,j}$ is associated with a volume fraction $v_{i,j}$, which is a scalar value in

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.

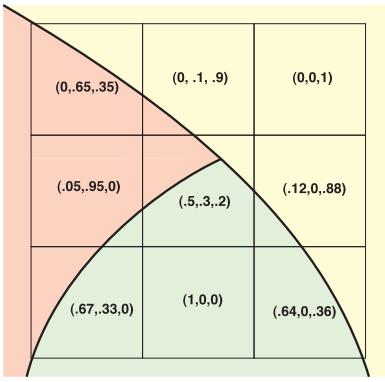


Fig. 1. The material interface problem: Three materials are distributed over a mesh, green, red, and yellow, respectively. One must reconstruct the interface between materials from only the given volume fractions, resulting in a (smooth) segmentation that accurately represents the given volumes per cell.

this case (in a two-material case, the second material has volume fraction $1 - v_{i,j}$), where $v_{i,j} = \{0, 1\}$ in pure cells, and $0 < v_{i,j} < 1$ in mixed cells.

Several previous efforts to reconstruct material interfaces attempt to define and approximate an $f(x, y)$ whose values represent a "volume fraction" at any point (x, y) (as opposed to only the cell centers) in the domain. From this function is defined an indicator function $\tau(x, y)$ that defines whether a position lies within a material or outside, where

$$\tau(x, y) = 0 \quad \text{or} \quad 1,$$

and $\tau(x, y)$ should provide a good approximation to the volume fraction v over any cell C, in that

$$\int_{(x,y)\in C} \tau(x, y) dxdy \approx v. \tag{1}$$

The boundary between 0 and 1 is usually defined as the material interface, and if the integral in (1) is an equality, then it is said that the volume fractions are preserved by the generated interface.

3.1 Summary

Our techniques make use of two novel concepts to compute f and τ. The volume fraction function f is constructed with the help of Coons patches, whereas the step function is defined by comparison between f and a smoothly varying isovalue field. The underlying concept of our approach is the following.

Given a volume fraction function $f(p)$, defined for any point p in two- or three-dimensional space, we define a spatially varying isovalue function $\sigma(p)$ using which we are able to set

$$\tau(p) = \begin{cases} 1 & \text{if } f(p) \geq \sigma(p) \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

Thus, the interface between $\tau = 0$ and $\tau = 1$, the material interface, is defined as the intersection between the volume fraction function and this smoothly varying isovalue function. Working with multiple materials, we construct a volume fraction function $f_i(p)$ and an isovalue function $\sigma_i(p)$ for each material i. Assuming, without loss of generality, that the materials are processed in order $1, \ldots, m$, the step function is then defined as

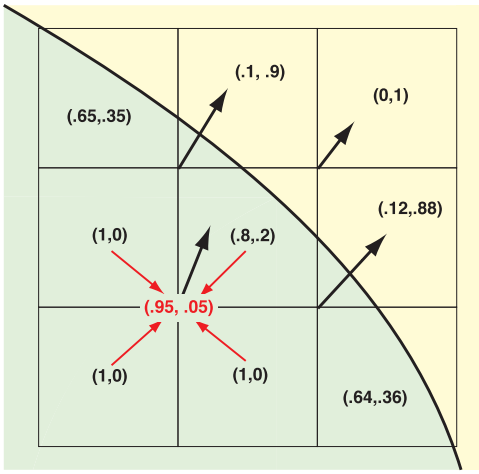


Fig. 2. Central differences are used to calculate the gradients at the corners of a cell. We note that the normal vector used in Youngs algorithm [6] is the normalized average of the gradient vectors calculated at the corners. Values of the volume fraction function at the cell corners are calculated by averaging the given cell-based volume fractions that contain the corner.

$$\tau_i(p) = \begin{cases} 1 & \text{if } f_i(p) \geq \sigma_i(p) \quad \text{and} \quad \forall j < i, \tau_j(p) = 0 \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

In the following, we detail the construction of f and σ and the resulting benefits to material interface reconstruction.

3.2 Volume Fraction Function

To construct a volume fraction function f that is continuous across cell boundaries, we compute higher order Coons patches based on the estimation of volume fraction gradients. Given a two-dimensional mesh, we can generate these gradient values at the corners of a cell by using central differences on the given cell-based volume fractions. We calculate function values at the corners of the cell by averaging the given cell-based volume fractions that contain the corner, see Fig. 2.

With these gradient values available, we can define the volume fraction function across each cell C as follows: by using the appropriate components of the gradients and the function values calculated at the endpoints of each edge of C, we compute scalar valued cubic Hermite curves [21] $v : \mathbb{R} \to \mathbb{R}$ along each edge as follows:

$$v(t) = v_0 H_0^3(t) + g_0 H_1^3(t) + g_1 H_2^3(t) + v_1 H_3^3(t) \tag{4}$$

$$H_0^3(t) = (1 - t)^2 (2t + 1) \tag{5}$$

$$H_1^3(t) = (1 - t)^2 t \tag{6}$$

$$H_2^3(t) = t^2 (3 - 2t) \tag{7}$$

$$H_3^3(t) = t^2 (t - 1) \tag{8}$$

$$0 \leq t \leq 1. \tag{9}$$

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.

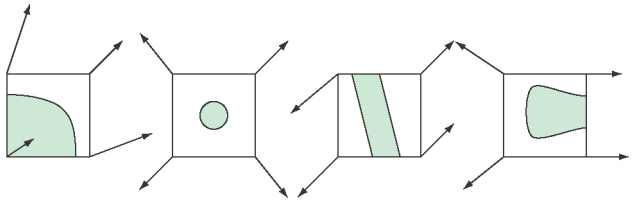


Fig. 3. A variety of interfaces that can be generated by using the gradient-based methods. Because of the gradient interpolation, a large class of interfaces is possible. Note that (c) and (d) are reconstructing the same volume fraction value.

In the above, v_0 and v_1 are the endpoint values and g_0 and g_1 are the gradients at the endpoints. Using cubic interpolation allows the interface to have up to three intersections per edge. This is an improvement over the majority of the existing methods. Given these four edge-based Hermite curves, we construct a bilinearly blended Coons patch [21], to represent a volume fraction function over C. Computing Coons patches for each cell results in f that is continuous over the entire mesh. In general, the values of f will lie near the interval $[0, 1]$ with some values outside the region in rare cases.

This approach allows a much greater variety of per-cell interfaces to be generated than is possible with lower order methods. It results in more topologically correct and volume accurate material interfaces, especially where fine perturbations are present in the interface. Fig. 3 illustrates a number of possible per-cell interfaces. Sample volume fractions that correspond to a few of the cases are shown in Figs. 6a and 6c. Many of these topologies cannot be reproduced with existing methods because they typically assume per-cell interface to be linear, or composed of a few triangles limited to at most one intersection on each edge. Other methods use linear interpolation and are unable to reproduce the likely curvature inside a cell.

The same method applies to three-dimensional meshes. We use central differences to calculate gradients at the corners of the cells, and average eight volume fractions from eight cells that contain a common corner, to create volume fraction function values at the corners. f_C is then computed in a similar fashion as in the two-dimensional case, utilizing three-dimensional trilinearly blended Coons function (see Coons Solid 2 in [22]).

3.3 Volume-Accurate Interface

Extracting a volume-accurate interface from f is done by assuming a discontinuous isovalue function σ_0 that defines a constant isovalue for each cell. We can calculate such σ_0 by defining that each cell C is independent of the entire mesh and deriving a per-cell isocontour of the volume fraction function that projects to a volume accurate interface within the cell. In general, f is a continuous, "smoothed," representation of the indicator function τ that we desire. The isovalue corresponding to this contour is chosen such that the projected area (in 2D) cut by this contour curve is equal to the given per-cell volume fraction v_C. (See Fig. 4 for an illustration of this process.) That is, we wish to select a constant isovalue k such that if Q is the region defined as all points with a per-point volume fraction value bigger than k

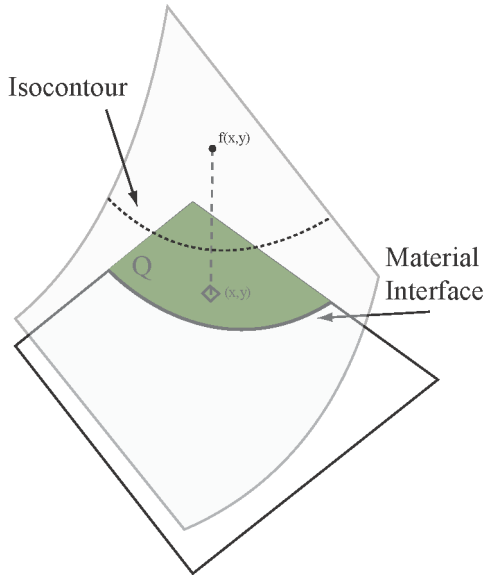


Fig. 4. Illustration showing the surface f_C, the isocontour generated by the value k, and the region Q, shown in green. The boundary of Q is defined to be the material interface.

$$Q = \{(x, y) \in C : f(x, y) >= k\}, \tag{10}$$

then $\mathrm{Area}(Q) = v_C$. In this isocontour problem, the known quantity v_C is the desired area of Q, and it is the determination of k, which is the difficulty. If we can determine the contour value k, then we define the boundary of Q to be the initial interface. We note that if all gradient vectors at the cell vertices are set to the gradient computed at the cell centroid, and f is planar, then this calculation approximates the method of Youngs [6]. If f_C is bilinear and we choose $k = \frac{1}{2}$ in (10), then we have the equivalent of Bonnell et al. [10].

3.3.1 Estimating Isovalues

In general, since f_C is a bicubic surface, the calculation of the contour value k is a difficult problem. Taking a Monte Carlo approach to this problem, we evaluate f_C at n_s uniformly sampled points $p_i, i = 1, \ldots, n_s$ over C, and consider the values $f_C(p_i)$, see Fig. 5. The choice of the number of samples n_s affects the accuracy of the reconstructed volume and is discussed in Section 3.5.1. Since f_C is a bicubic surface, generating the values $f_C(p_i)$ is a straightforward iterative procedure (see [21]) containing only a few adds and multiplies per step.

To find an approximation to the region Q, we need to partition these points into two sets S_1 and S_2, such that

$$p \in S_1 \quad \text{and} \quad p' \in S_2 \Rightarrow f(p) > f(p'), \tag{11}$$

and

$$\frac{\|S_1\|}{n_s} \approx v_C \tag{12}$$

$$\|S_1\| \approx n_s \times v_C. \tag{13}$$

Given (11) and (13), we can conclude that the greatest $k = v_C \times n_s$ samples are in S_1. To find set S_1, we compute the value of the kth sample and set $S_1 = \{p_i, f(p_i) \geq k\}$.

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.

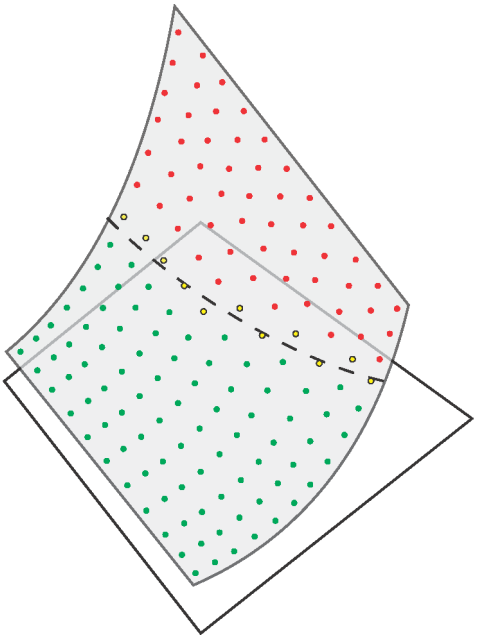


Fig. 5. Generating n_s points on f_C, we use nth-element search to find a threshold such that the fraction of the points above the threshold matches the given volume fraction. Samples are regularly spaces for illustration purposes. In our method, they are distributed randomly.

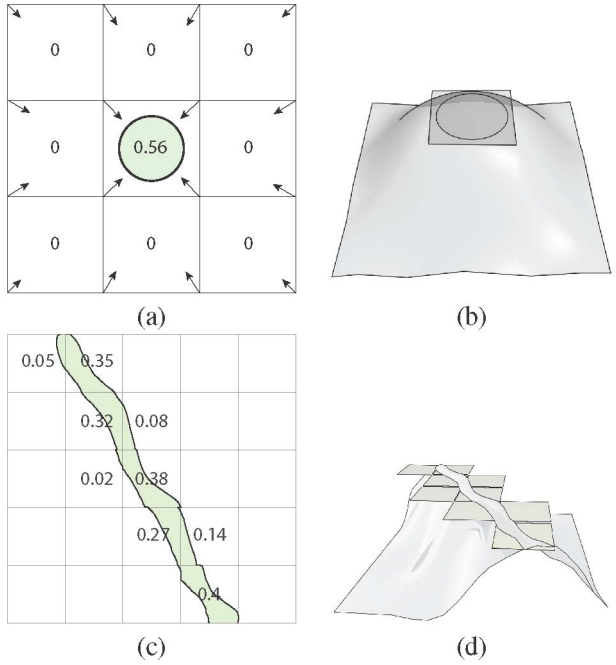


Fig. 6. Sample 2D interfaces produced by the volume-accurate method. The volume fractions and gradients are overlayed in (a) and (c). The corresponding visualizations of the volume fraction function are in (b) and (d).

This is achieved with a simple nth-element selection algorithm, which runs in the worst-case linear time (and space) (see [23]).

As a result, we model the volume fraction field with f and the isovalue field with a piecewise constant σ_0. Projection of the per-cell isocontours onto the 2D plane results in a volume accurate interface. In Fig. 6a, we are reconstructing a circle that has been sampled onto a grid. The volume fraction function is visualized in Fig. 6b. Since both the gradients and the volume fractions are symmetric, it looks like a Gaussian kernel. In this simple case, the resulting interface closely approximates a circle. An example of interesting topology is presented in Fig. 6c. Here, the challenge is to reconstruct a thin stripe of material that is not aligned with coordinate axes. A visualization of the volume fraction function for this situation is shown in Fig. 6d. This a good example of the intuition behind our method. The volume fraction function has higher values in areas where the material is more likely to be present. Per-cell isovalues ensure that the volume fractions are preserved, while the cubic nature of volume fraction function gives the interface curved shape, which is likely to occur in fluid simulations. A drawback of the volume-accurate method is that it might cause frequent sharp changes in interface normals across cell boundaries as is visible in Fig. 6c. We discuss how to compute a continuous isovalue field σ and use it to generate a smooth interface in Section 4.

3.4 Multiple Materials

Our method extends naturally to multimaterial data sets. When we have only two materials, we use a single volume fraction function $f(x, y)$ that model the change in the volume of the first material over the data set. Using the isovalue field σ, we compute the space occupied by the first material, while any unoccupied space is left for the second material. The approach we use for multiple materials is a generalization of this technique. Given n materials, we generate $n-1$ volume fraction functions $f_i(x, y)$ that each model the input volume fractions that belong to material i. To generate the material interfaces, we need to compute $n-1$ isovalue fields as well. We compute the per-cell isovalues for each of the materials. In a given cell C, each present material receives an isovalue and claims some portion of C. We iterate over the materials and only take into consideration the unclaimed area of C. In each cell, we generate n_s uniformly sampled points. Let us define P to be the set of the sampled points. The following iterative procedure then determines the isovalues λ_i for each of the materials:

$R \leftarrow P$
for $i \in 1..n-1$ **do**
  $\lambda_i \leftarrow$ select $(v_{i,C} \times n_s)$th largest $f_i(p_i)$ from R
  $S_i \leftarrow \{p_i : f_i(p_i) \geq \lambda_i\}$
  $R \leftarrow R \setminus S_i$
end for

We start with a set R of n_s generated samples. Then, for each material we evaluate the points in R and use the selection process to find the desired number of largest samples in R that approximate the volume $v_{i,C}$. The samples are evaluated with the function $f_{i,C}$ that corresponds to the material in question. The set of selected points is then removed from R, and the procedure is repeated for the next material. The computed $\lambda_i, i \in \{1, \ldots, n\}$ are the isovalues that correspond to the volume accurate interfaces for each material in a cell.

Note that this induces an order-dependent reconstruction. However, the order in which the materials are processed does not affect the reconstruction accuracy. The primary factor that affects the reconstructed volume is the density of the samples we use to compute the isovalues. The resolution with which we can compute these isovalues is uniform throughout the cell, so each material's volume will be reconstructed with the same precision. The order can, however, affect the shape of

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.

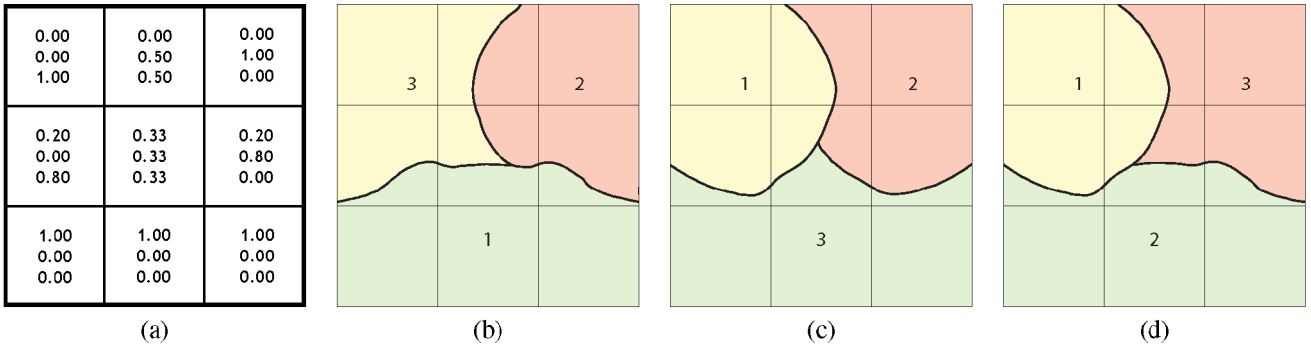


Fig. 7. Classic t-junction-style 3×3 data set is given in (a). Reconstructions resulting from different orders of processing materials in (b), (c), and (d). The full data set is shown, i.e., the next ring of cells are empty. The processing orders affect the concavity of the interface in both two- and three-material cells.

the interface, mainly the concavity. Each material carves out a portion of a cell, which, by the nature of cubic interpolation, produces cavity. An example of interfaces resulting from different material processing orders is shown in Fig. 7. At the same time, the problem is inherently underconstrained, which means that there is not a single "correct" order of processing materials. We recommend selecting an order that best fits the physical properties of the interacting materials.

3.5 Additional Notes

In this section, we will discuss the accuracy metrics used in the rest of the paper and then show how the sampling parameter n_s affects the reconstruction. One of the primary metrics of a material interface reconstruction algorithm is the reconstructed volume error. Note that we will use the terms "volume" and "volume fraction" interchangeably as we can assume each cell has a unit volume, in which case the given volume fractions indeed can be thought of as the volume occupied by a material. It is necessary for a material interface reconstruction method to preserve the input volume fractions accurately to have confidence that the resulting interface is a valid solution to the problem.

3.5.1 Sampling

A key parameter that affects the accuracy of the initial per-cell isovalues is the number of samples used in the discrete approximation described in Section 3.3.1. The key feature of the proposed interface approximation scheme is that it is volume-accurate in the limit, i.e.,

$$\lim_{n_s \to \infty} \frac{\|S_1\|}{n_s} = v_C. \tag{14}$$

Similarly to Monte-Carlo sampling, a larger number of samples n_s will provide more input for the selection algorithm, minimizing the difference between the selected sample value and the actual isovalue, which corresponds to the volume-accurate interface. Fig. 9 shows the reconstruction error of a single 3D cell for various volume fractions using $n_s = 3{,}000$ and $n_s = 6{,}000$. The jaggedness of the curve is due to the random nature of sampling, which occasionally allows a better approximation to be found using fewer cells. In general, however, a larger n_s has smaller reconstruction error, especially when the input volume is small relative to the size of the cell.

In practice, the number of samples is limited by the available memory, because the sample values need to be stored to perform nth element selection. Performance is another issue, especially in SIMD style implementations, because the nth element search is prone to thread divergence. Fig. 8 shows sample timing data for various number of samples. The performance decreases linearly with the increasing number of samples evaluated per cell as expected.

Decreasing n_s causes difficulty in matching smaller volumes but allows for higher performance and the ability to process larger data sets. Since we desire real-time performance, we typically use $n_s = 3{,}000$.

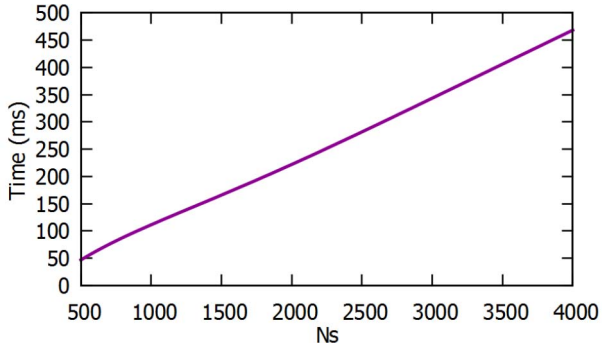


Fig. 8. Time taken to perform a discrete approximation to the volume-accurate per-cell isovalues on an NVIDIA GTX 580 video card. The data set contains 7,130 mixed cells.

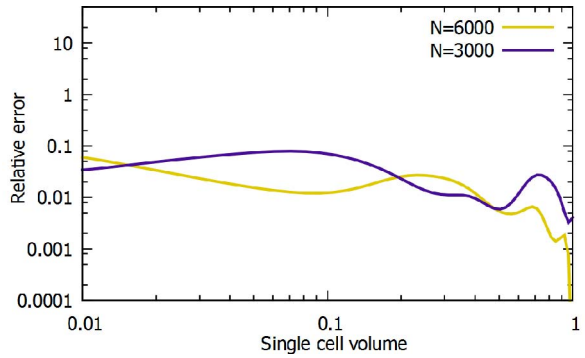


Fig. 9. Relative error of the reconstructed volume for a single cell plotted against a range of input volume fractions. N is the number of samples used to derive the isovalue in the discrete isovalue estimation from Section 3.3.1.

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.

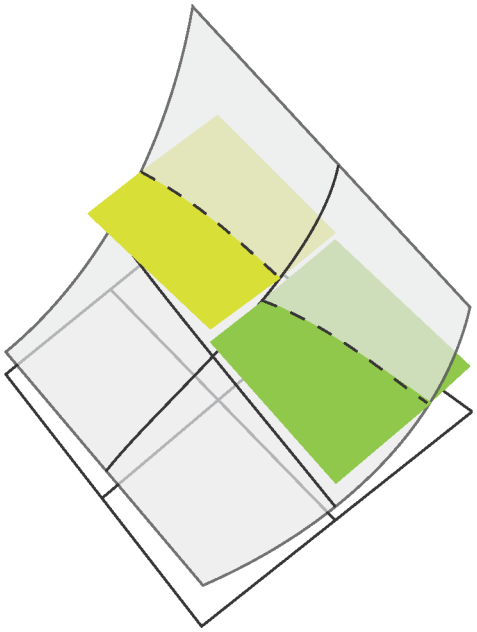


Fig. 10. Interface approximation corresponding to the constant per-cell isovalues. The interface has rough transitions across the cell boundaries. Effectively the isovalue curves representing the boundaries are generated by intersecting a plane (shown in green or yellow in the figure) with the volume fraction function f_C over the cell C, generating an isovalue curve that induces a boundary with the correct volume fraction. Adjacent cells have different volume fractions and, thus, continuity across the cell edges cannot be guaranteed.

3.5.2 Error Metric

We compute the reconstructed material volume using Monte-Carlo sampling. The number of samples n_a we use for volume estimation has a similar effect on volume approximation as n_s has on computing a correct isovalue. However, error estimation can be done offline and we set n_a to a higher number. We have empirically found the necessary number of samples $n_a = 9{,}000$. Increasing n_a beyond this number has insignificant impact on the estimation of the reconstructed volume.

The error metric we use is computed as the relative difference between the reconstructed and the given volumes. By setting $v'_{m,C}$ to be the reconstructed volume, the per-cell error for a single material m becomes

$$e_{m,C} = \frac{\|v'_{m,C} - v_{m,C}\|}{v_{m,C}}. \tag{15}$$

The error is always higher for the cells with smaller volumes of material, in the relative sense. We capture this fact in a histogram that computes a metric for separate intervals of given volume. We partition the unit volume into 10 intervals (bins) and for each interval, compute the median of errors that arise when attempting to reconstruct a volume that falls into that interval. More formally, we have volume intervals V_i, $i = 0, \ldots, 9$, where each V_i represents an interval of $[i \cdot 0.1, (i+1) \cdot 0.1]$. Each interval is a set containing reconstructed volume errors:

$$V_i = \{e_{m,C},\ i \cdot 0.1 \leq v_{m,C} < (i+1) \cdot 0.1\}. \tag{16}$$

We then report the median of all $e_{m,C} \in V_i$ for each V_i in a histogram. We use this method to report reconstruction errors in Sections 4 and 6.

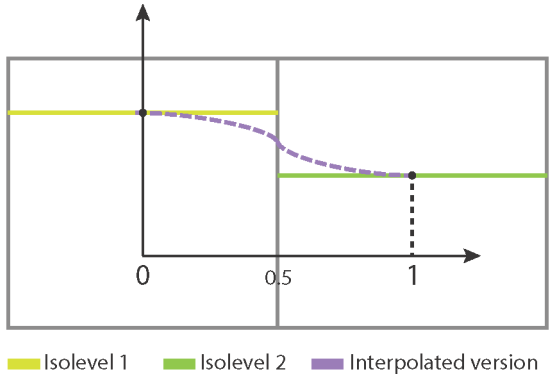


Fig. 11. Cross section of the cells in Fig. 10. The piecewise-constant isovalue function is smoothed by interpolating isovalues in neighboring cells (in purple).

4 Isovalue Function

As stated earlier, our approach makes direct use of an isovalue function σ to define the indicator function (and thus the final interface). Using this second spatially varying function to define the material interface has several beneficial properties, such as easy control over reconstruction accuracy and smoothness.

Thus, the step of defining such a continuous isovalue function is an important part to our interface reconstruction scheme. The final value for $\sigma(p)$ is computed via a multiple stage process. First we compute $\sigma(p)$ at the centroid of each cell (see Section 3.3). The centroid values are then interpolated across the mesh as needed. The applied interpolation method defines the balance between accuracy and smoothness of the interface.

The volume-accurate interface derived using the isovalue function σ_0 from Section 3.3 may have sharp transitions across the cell boundaries. The per-cell isovalues of σ_0 are generated from the volume fractions, and with differing volume fractions in adjacent cells, smoothness cannot be ensured (although in practice, the sharp transitions are generally small), see Fig. 10. We can provide control over accuracy and physical plausibility of the interface by blending the planes defined by constant per-cell isovalues across the edges (or faces in 3D) of the cells, creating a smoothly varying isovalue field across the entire mesh. A schematic representation of the blending is shown in Fig. 11. By modifying isovalues across the cells, we compromise the volume-fraction preserving nature of the constant per-cell approximations, but can use this to enhance normal approximations of the interface.

Given four adjacent cells (in 2D), and a point on each plane corresponding to the center of the cell, we can interpolate the centroid values to produce a surface. This piecewise surface now intersects the volume fraction function in a "boundary" that is generally continuous over the edges of the cells. Once the planes for each cell have been defined, this modification can be integrated directly into the visualization process (see Section 5), and continuous visualizations produced. We use an interpolating function that allows us to vary the results between accurate volume reconstruction and quality visualizations.

By using a smoothstep function [24] to interpolate between the per-cell isovalues, we gain the ability to control how sharp the transition is between two neighboring cells near their boundary (see Fig. 4). Additionally, this ensures that the

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.

isovalue at the cell boundary is the same in the neighboring cells, which forces per-cell interfaces to meet at a single point. This, in turn, allows us to balance the accuracy with the plausibility of the interface. Sharper step function results in a rough transition of the interface across cell boundaries, closely matching our volume-accurate interface that uses discontinuous σ_0 described in Section 3.3. Our implementation uses the following smoothstep function h:

$$h(x) = \begin{cases} 0, & x \leq 0 \\ \frac{27}{6}x^3 & 0 < x \leq \frac{1}{3} \\ -\frac{5}{8} + \frac{9}{4}x - \frac{1}{3}\left(3x - \frac{3}{2}\right)^3 & \frac{1}{3} < x \leq \frac{2}{3} \\ 1 - \frac{1}{6}(3 - 3x)^3 & \frac{2}{3} < x \leq 1 \\ 1 & x > 1. \end{cases} \tag{17}$$

Here, x is the interpolation offset $0 \leq x \leq 1$ and t is the parameter that affects the smoothness—accuracy balance of the interface. By setting t to a small positive value close to zero, the function closely approximates the discontinuous step function. Increasing t results in a smoother transition between the values, approaching linear interpolation in the limit, see Fig. 13.

When t is close to zero, per-cell interfaces contribute to rough boundaries between the cells. This causes the unwanted artifacts to appear in the image. However, when it is desired to capture all occurrences of a material in the data set, this setting is most appropriate. Setting t to a larger value will cause the artifacts to disappear at the cost of higher reconstruction error, especially for cells with small amount of material. For a relatively high value of t, the interface is smooth and provides physically more plausible interface normals, although some fine detail may no longer be present. These effects are shown in Fig. 12. The corresponding median reconstruction errors are reported per 10 volume intervals as described in Section 3.5.2. Note that even a high level of smoothing maintains comparable reconstruction error to lower smoothing parameters. This is visible in Fig. 12a.

With these computations, we are able to reconstruct the desired material interface by intersecting the isovalue field with the volume fraction field on a per cell basis. For easy control over volume interface smoothness, we allow the per-cell isovalues to be interpolated across cells by a variable smooth-step function. Hereby, nearest neighbor interpolation (i.e., a true step function) corresponds to constant isovalues per cell, whereas higher-order smooth step functions increase smoothness of the interface while sacrificing reconstruction accuracy within the cells. This provides us with a single parameter for direct control over accuracy and smoothness—a parameter controlling the smoothness of the smooth step function used for isovalue interpolation.

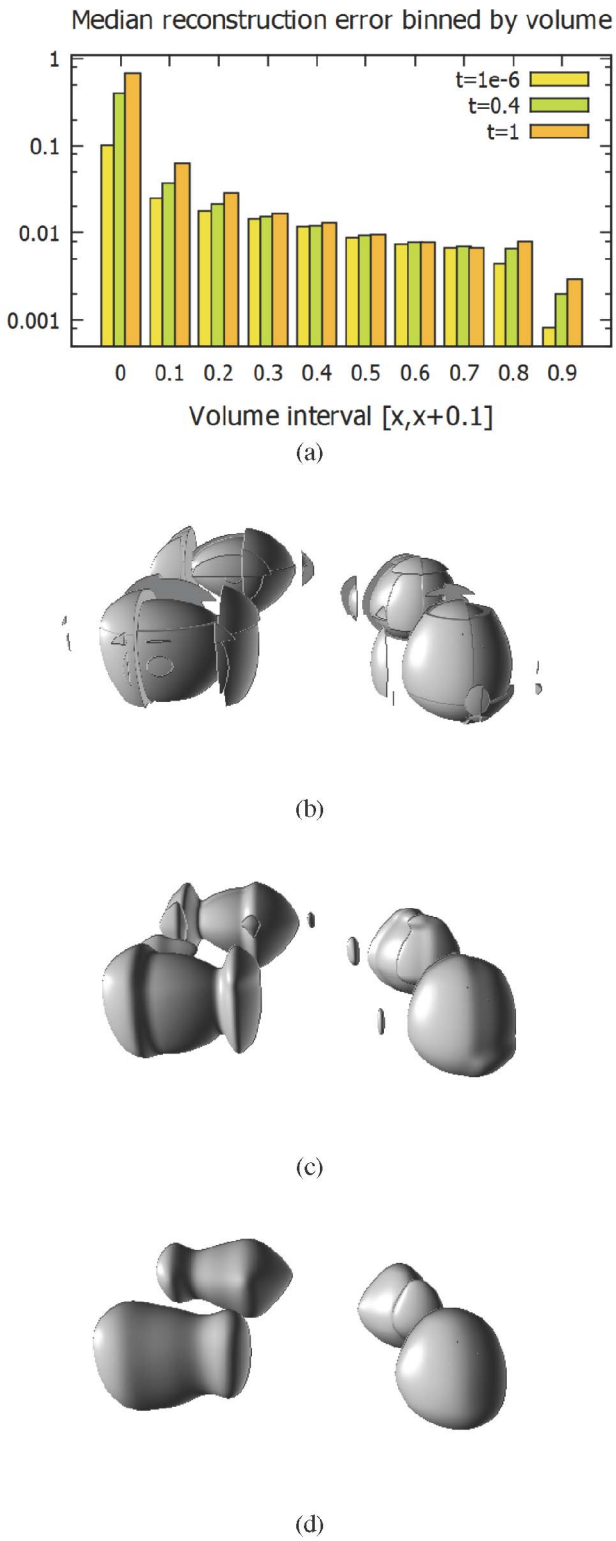


Fig. 12. Demonstrating the effect of the smoothing parameter t on a bubble simulation. (a) Median reconstruction errors for different values of t. (b) When t is close to 0, per-cell interfaces contribute to rough boundaries between the cells. (c) As t becomes larger, the artifacts disappear. (d) For a relatively high value of t the interface is smooth, although some fine details are not present.

5 Implementation

There are effectively two stages in our method. First, we compute the per-cell constant isovalues as described in Section 3.3.1. These isovalues are computed on the GPU, parallelizing over data with each thread dealing with a single cell. The constant isovalues computed at the centroids of each cell form an isovalue grid. This grid is then stored on the device for fast access during rendering. The visualization stage casts rays into the original mesh (cells) stored in a bounding volume hierarchy. Inside each cell a ray hits, we can step through it, evaluate the isovalue $\sigma(p)$ and the volume fraction function $f(p)$ and report an intersection if

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.

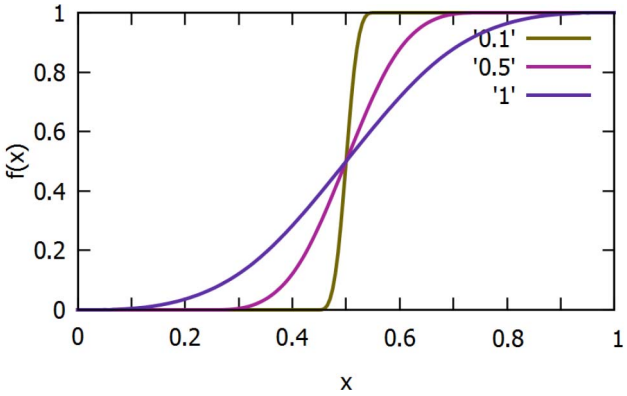


Fig. 13. Smooth-step function with different parameters t. When t is close to zero, $h(x)$ approximates a step function. As t increases, it first transforms to a smooth-step and then approaches to a linear function.

$f(p) \geq \sigma(p)$. The interpolation method for smoothing is applied at each evaluation point p. This is similar to the work of Hadwiger et al. [25]. Surface normals can be found analytically by differentiating the function $f(p) - \sigma(p)$. We approximate the normal at the intersection point by combining intersection information from the neighboring pixels. By forming a triangle from three points on the interface that were found by tracing rays through three neighboring pixels, we can use the normal of the triangle as a reasonable approximation of the surface normal.

If a discrete geometry representation is desired, the preprocessing data, namely $f(p)$ and $\sigma(p)$ can be used in Marching Cubes family techniques that are able to handle spatially varying isovalues.

We take advantage of the simplicity and scalability of the NVIDIA OptiX engine [26] to aid in GPU implementation and parallelization of the visualization stage.

6 Results

6.1 Model Data

To analyze the ability of the proposed algorithm to reconstruct known surfaces, we present the output for synthetic data. The first data set is a simple tetrahedron, with nonaxis-aligned faces sampled to a $32 \times 32 \times 32$ grid of volume fractions. The reconstruction is shown in Fig. 14. The ripples across the edges of the tetrahedron are due to preference for high curvature surfaces induced by cubic interpolation. This is unavoidable when the resolution is so low. The input volume fractions do not contain enough information to reconstruct correct topology, i.e., the ground-truth data could contain ripples and still be represented by the same volume fractions. However, the low-sampling resolution does not prevent the computed gradients from capturing the linearity of the faces.

The second data set is a bonsai tree, which is shown in Fig. 15. The original surface was extracted from a $256 \times 256 \times 256$ scalar field. The ground-truth mesh is an isosurface of the scalar field, which is then sampled to get a lower resolution volume fraction data. Some of the flat leaves of the tree acquire more curvature, but despite the low-resolution volume fraction field ($64 \times 64 \times 64$), the organic shape of the tree is well reproduced. As the resolution increases, the shape will eventually converge to the original.

Fig. 14. Reconstructed tetrahedron from volume fraction data sampled from mesh to a $32 \times 32 \times 32$ grid.

6.2 Practical Data

We evaluate our method on several real-world data sets and compare it with the iterative version of the method proposed by Meredith and Childs ("Equi-Z/i5")[12]. We use VisIt's[27] implementation of EquiZ/i5 with five iterations of adaptive refinement and default convergence ratio of 0.4. This algorithm appears to be the most competitive out of the existing ones in terms of the tradeoff between quality of the interface and performance [12]. It has a low reconstruction error maintained by adaptive refinement iterations and good performance due to a table lookup-style implementation.

The data sets presented here have been generated from CFD simulations. The first one is a two-material simulation with an air bubble rising from a viscous fluid and breaking on the surface. This is $64 \times 64 \times 64$ data set with the vast majority of the mixed cells having the amount of material $v_C > 0.1$. The comparison with EquiZ/i5 is presented in Fig. 16. The cubic gradient-based interface provides much more detail and does a better job representing fine elements. It also provides a more realistic look—this resembles an actual snapshot of a fluid simulation, as opposed to the rough, blocky approximation maintained by EquiZ/i5.

The second data set we will refer to as "Splash." It is a three-material rain simulation with a large drop landing into an oil tank, resulting in a mix of the two fluids splashed all over the data set. The third material is air, which is transparent to make all interfaces visible. The reconstruction of the material interface from the simulation progress at two different time steps are shown in Fig. 17. The data set size is $80 \times 80 \times 80$. Time steps 12 and 21 are shown, containing 166,780 and 353,462 mixed cells, respectively.

In the comparison runs for our method, we use $n_s = 3{,}000$ samples for the discrete isovalue computation and set the smoothing parameter $t = 1$—giving higher preference to the realistic topology of the interface. This is especially beneficial when dealing with fluid simulations.

Fig. 19 shows the reconstruction error comparison with different values for the smoothing parameter t. The figure shows the median of the errors for each volume interval as described in Section 3.5.2. The final interface produced by our method is based on the volume-accurate estimation.

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.

(a) (b)

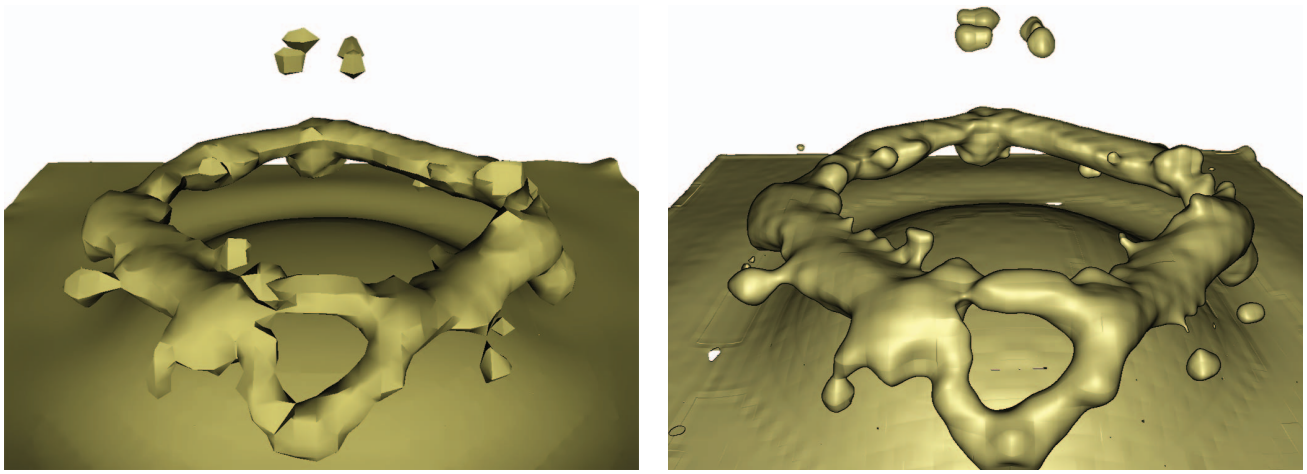
Fig. 15. (a) Ground truth data—isosurface from a scalar field with resolution $256 \times 256 \times 256$. (b) Interface reconstruction from volume fraction data with resolution is $64 \times 64 \times 64$.

Even though some error is acquired by the isovalue interpolation, it is fairly small in practice, as demonstrated by the height pattern similar to the one in Fig. 12a. The result obtained by performing the reconstruction with smoothing parameter $t = 1$ shows larger deviation from $t = 0.4$ and $t = 0.001$ for $v_C < 0.4$ but the reconstruction error is still much smaller than in EquiZ/i5.

As mentioned earlier the order of processing materials affects only the shape of the interface and not the accuracy of the reconstruction. The error data for only the cells containing three materials is presented in Fig. 18. Several different processing orders result in similar overall reconstruction error, as expected.

It is important for interface reconstruction algorithms to be able to identify features such as small bubbles, thin rods, or other very fine detail. These fine elements present a challenge to most existing algorithms. If such fine elements occur independently in the domain, separated from other cells with the same material, our method approximates them well, because the gradient field of the volume fraction function will indeed indicate that these fine elements are present and the produced interface will preserve the volume fractions well because there is little noise from surrounding cells. The Splash data set has many small bubbles scattered throughout space and as can be seen from the results, the proposed method captures them well.

Another significant contribution of our method is that it is not as sensitive to the resolution of the mesh as existing methods. This is an important feature as it allows for larger simulations to be employed for analysis maintaining good visual quality. Curvature, derived from the Coons macroelements, along with just-in-time rendering, allows the interface to retain smooth shape, more natural for fluid simulations. Mesh-based algorithms, on the other hand, are typically limited to a few triangles per cell, which impedes the quality of the interface as the data set resolution becomes more coarse. For example, it would



(a) (b)

Fig. 16. A two-material bubble simulation $64 \times 64 \times 64$ data set with 7,130 mixed cells. (a) EquiZ/i5 output side-by-side comparison with our method in (b).

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.

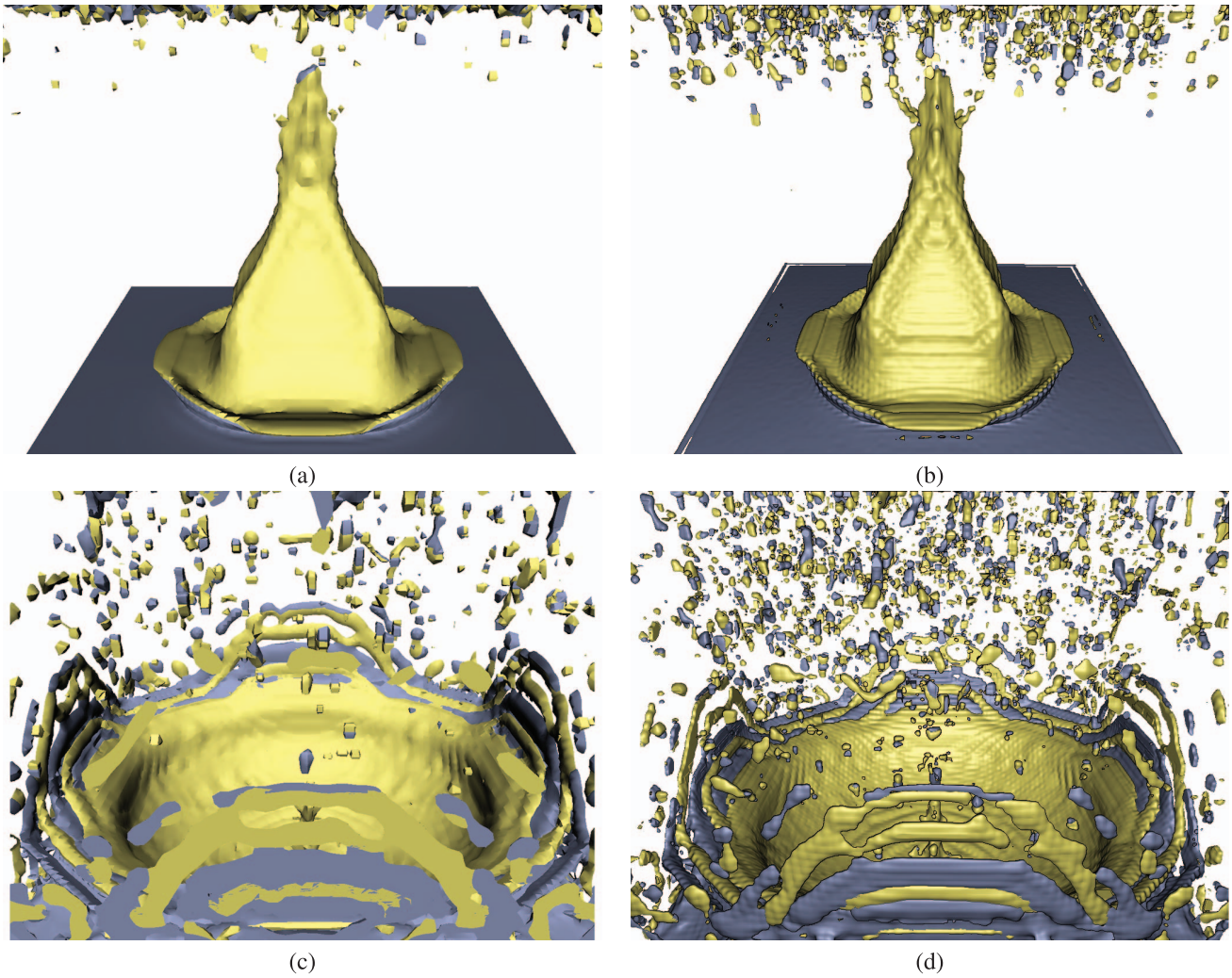


Fig. 17. A three-material CFD simulation $80 \times 80 \times 80$ data set with 353,462 mixed cells. EquiZ/i5 output is in (a) and (c) side-by-side comparison with our method in (b) and (d).

take a significantly higher resolution of the input mesh for Equi-Z/i5 to deliver results comparable to our method.

We provide the timing data for both algorithms ran on the Splash simulation data set in Table 1. The hardware used was an NVIDIA GTX 580 video card. This data are presented as a general guideline. The GPU implementations are typically sensitive to device-specific optimizations. On the other hand, most of the time is spent in computing per-cell isovalues—a process that only accesses one ring of neighbor cells and exhibits potential for better load balancing further improving performance. We note that random number generation for the Monte-Carlo section of

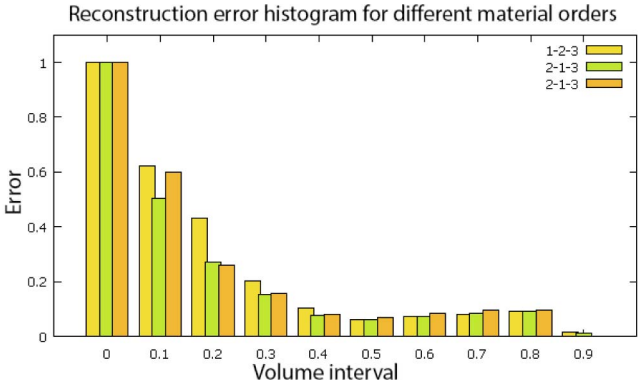


Fig. 18. The relative reconstruction error is plotted for the splash simulation at time step 21 for different orders of processing materials. The median of errors is shown for each volume bucket for only the cells containing all three materials.

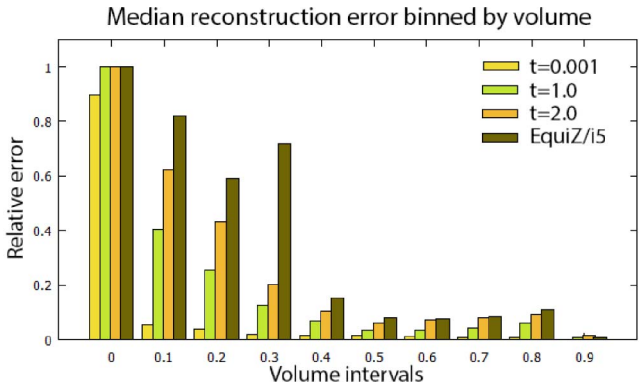


Fig. 19. Relative reconstruction error is plotted for the splash simulation at time step 21. The median of errors for each volume bucket is shown.

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Timing Data for the Splash Data Set

Mixed Cells	166780	353462
Gradient-based method timing (sec)		
Computing Gradients	0.702	0.749
Discrete Isovalues	9.001	13.026
GPU I/O	2.199	4.930
Rendering (1024x768)	1.747	3.136
EquiZ/i5 (VisIt) timing (sec)		
Full reconstruction	6.6362	8.284

the computation takes a negligible amount of time compared to the evaluation and selection of samples. EquiZ/i5 algorithm is faster in general and while the initial stage of the algorithm can also be parallelized, most of the time is spent in the adaptive refinement, which is more difficult to implement in parallel.

7 Conclusion

We have described a new algorithm for material interface reconstruction. This method is based upon a gradient-based technique of computing a continuous volume fraction function and a continuous isovalue field. With an approximation to a volume-accurate per-cell interface at the foundation of the method, the volume fractions are preserved well. Using a cubic function to model the volume fraction function enables many frequently occurring interface topologies, that are not possible with existing methods. Manipulating the interface with a continuous isovalue field allows for control between accuracy of the reconstructed volume fractions and realistic interface topology. The method is straightforward to implement, and is based upon robust principles. It has good performance and exhibits robustness to low resolution input.

There are several directions that we would like to explore in continuation of this work:

- Investigate other options for cubic interpolation methods and gradient approximation methods and their effect on the interface. The method presented here could potentially be used as a framework for a variety of custom methods, tuned for specific types of data.
- Further, study the possibility of improving the load balancing on GPU and implementing the algorithm on a GPU cluster.
- Integrate this method into those problems that utilize additional per-cell information (e.g., face fractions, edge fractions, or material centroids) as well as add support for more grid types.

Acknowledgments

This work was supported in part by the National Science Foundation under contracts IIS 0916289 and IIS 1018097, and the Office of Advanced Scientific Computing Research, Office of Science, of the US Department of Energy under Contract No. DE-FC02-06ER25780 through the "Scientific Discovery through Advanced Computing" (SciDAC) programs Visualization and Analytics Center for Enabling Technologies (VACET), and contract DE-FC02-12ER26072, entitled Scalable Data Management, Analysis, and Visualization (SDAV) Institute. The authors thank the members of the Institute for Data Analysis and Visualization at UC Davis for their support.

References

[1] W.E. Lorensen and H.E. Cline, "Marching Cubes: A High Resolution 3D Surface Construction Algorithm," *Computer Graphics,* vol. 21, no 4, pp. 163-170, July 1987.
[2] H.-C. Hege, M. Seebaß, D. Stalling, and M. Zöckler, "A Generalized Marching Cubes Algorithm Based on Non-Binary Classifications," technical report, Zuse-Institut Berlin, 1997.
[3] Z. Wu and J.M. Sullivan, "Multiple Material Marching Cubes Algorithm," *Int'l J. Numerical Methods in Eng.,* vol. 58, no. 2, pp. 189-207, http://onlinelibrary.wiley.com/doi/10.1002/nme.775/abstract, 2003.
[4] D.C. Banks and S. Linton, "Counting Cases in Marching Cubes: Toward a Generic Algorithm for Producing Substitopes," *Proc. IEEE 14th Visualization 2003 (VIS '03),* pp. 8-15, http://dx.doi.org/10.1109/VISUAL.2003.1250354, 2003.
[5] G.M. Nielson and R. Franke, "Computing the Separating Surface for Segmented Data," *Proc. IEEE Visualization,* pp. 229-233, 1997.
[6] D.L. Youngs, "Time-Dependent Multi-Material Flow with Large Fluid Distortion," *Numerical Methods for Fluid Dynamics,* K.W. Morton and J.J. Baines, eds., pp. 273-285, Academic Press, 1982.
[7] D. Gueyffier, J. Li, A. Nadim, R. Scardovelli, and S. Zaleski, "Volume-of-Fluid Interface Tracking with Smoothed Surface Stress Methods for Three-Dimensional Flows," *J. Computational Physics,* vol. 152, pp. 423-456, 1999.
[8] J.E. Pilliod and E.G. Puckett, "Second-Order Accurate Volume-of-Fluid Algorithms for Tracking Material Interfaces," technical report, Lawrence Berkeley Nat'l Laboratory, 2000.
[9] K.S. Bonnell, D.R. Schikore, K.I. Joy, M. Duchaineau, and B. Hamann, "Constructing Material Interfaces from Data Sets with Volume-Fraction Information," *Proc. IEEE Visualization '00,* pp. 367-372, Oct. 2000.
[10] K.S. Bonnell, M.A. Duchaineau, D.R. Schikore, B. Hamann, and K.I. Joy, "Material Interface Reconstruction," *IEEE Trans. Visualization and Computer Graphics,* vol. 9, no. 4, pp. 500-511, Oct.-Dec. 2003.
[11] G. Treece, R. Prager, and A. Gee, "Regularised Marching Tetrahedra: Improved Iso-Surface Extraction," *Computers and Graphics,* vol. 23, no. 4, pp. 583-598, 1999.
[12] J. Meredith and H. Childs, "Visualization and Analysis-Oriented Reconstruction of Material Interfaces," *Computer Graphics Forum,* vol. 29, no. 3, pp. 1241-1250, 2010.
[13] J.C. Anderson, C. Garth, M.A. Duchaineau, and K. Joy, "Discrete Multi-Material Interface Reconstruction for Volume Fraction Data," *Computer Graphics Forum,* vol. 27, no. 3, pp. 1015-1022, http://graphics.cs.ucdavis.edu/cobalt, 2008.
[14] F. Wu, "The Potts Model," *Rev. Modern Physics,* vol. 54, no. 1, pp. 235-268, 1982.
[15] P. Feng, T. Ju, and J. Warren, "Piecewise Tri-Linear Contouring for Multi-Material Volumes," *Proc. Sixth Int'l Conf. Advances in Geometric Modeling and Processing (GMP '10),* pp. 43-56, http://dx.doi.org/10.1007/978-3-642-13411-1_4, 2010.
[16] J.C. Anderson, C. Garth, M.A. Duchaineau, and K.I. Joy, "Smooth, Volume-Accurate Material Interface Reconstruction," *IEEE Trans. Visualization and Computer Graphics,* vol. 16, no. 5, pp. 802-814, Sept./Oct. 2010.
[17] M. Kass, A. Witkin, and D. Terzopoulos, "Snakes: Active Contour Models," *Int'l J. Computer Vision,* vol. 1, no. 4, pp. 321-331, 1988.
[18] M. Kucharik, R.V. Garimella, S.P. Schofield, and M.J. Shashkov, "A Comparative Study of Interface Reconstruction Methods for Multi-Material ALE Simulations," *J. Computational Physics,* vol. 229, pp. 2432-2452, Apr. 2010.
[19] V. Dyadechko and M. Shashkov, "Reconstruction of Multi-Material Interfaces from Moment Data," *J. Computational Physics,* vol. 227, pp. 5361-5384, http://portal.acm.org/citation.cfm?id=1365099.1365338, May 2008.

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.

[20] S.P. Schofield, R.V. Garimella, M.M. Francois, and R. Loubère, "A Second-Order Accurate Material-Order-Independent Interface Reconstruction Technique for Multi-Material Flow Simulations," *J. Computational Physics,* vol. 228, pp. 731-745, Feb. 2009.
[21] G. Farin, *Curves and Surfaces for CAGD: A Practical Guide,* fifth ed. Morgan Kaufmann Publishers Inc., 2002.
[22] R. Durikovic and S. Czanner, "Modelling with Three Types of Coons Bodies," *Int'l J. Modelling and Simulation,* vol. 24, no. 2, pp. 1-5, 2004.
[23] T.H. Cormen, C. Stein, R.L. Rivest, and C.E. Leiserson, *Introduction to Algorithms,* second ed. McGraw-Hill Higher Education, 2001.
[24] Q. Li, "Blending Implicit Shapes Using Smooth Unit Step Functions," *Proc. Ninth ACM Symp. Solid Modeling and Applications,* 2004.
[25] M. Hadwiger, C. Sigg, H. Scharsach, K. Bhler, and M.H. Gross, "Real-Time Ray-Casting and Advanced Shading of Discrete Isosurfaces," *Computer Graphics Forum,* vol. 24, no. 3, pp. 303-312, http://dblp.uni-trier.de/db/journals/cgf/cgf24.html#HadwigerSSBG05, 2005.
[26] S.G. Parker, J. Bigler, A. Dietrich, H. Friedrich, J. Hoberock, D. Luebke, D. McAllister, M. McGuire, K. Morley, A. Robison, and M. Stich, "Optix: A General Purpose Ray Tracing Engine," *ACM Trans. Graphics,* vol. 29, pp. 66:1-66:13, http://doi.acm.org/10.1145/1778765.1778803, July 2010.
[27] H. Childs, E.S. Brugger, K.S. Bonnell, J.S. Meredith, M. Miller, B.J. Whitlock, and N. Max, "A Contract-Based System for Large Data Visualization," *Proc. IEEE Visualization '05,* pp. 190-198, 2005.

Iuri Prilepov received the bachelor's degree in computer science from the University of California, Davis, and is currently a doctorate student at the Institute for Data Analysis and Visualization of UC Davis mainly doing research in material interface reconstruction.

Harald Obermeier received the diploma and PhD (Dr rer nat) degrees in computer science from the University of Kaiserslautern, Germany in 2008 and 2011, respectively. He is currently a postdoctoral researcher at the Institute for Data Analysis and Visualization at the University of California Davis. His research interests lie in the scientific visualization of (multiple) fields with applications in areas such as continuum mechanics and geology. He is a member of the IEEE Computer Society.

Eduard Deines received the MSc (Dipl-Inform) and PhD (Dr rer nat) degrees in computer science from the University of Kaiserslautern in 2004 and 2008, respectively. Thereafter, he was a postdoctoral researcher with the Institute for Data Analysis and Visualization at the University of California, Davis. During his time in Kaiserslautern and Davis, his research focused on scientific visualization in room acoustics, flow visualization, and surface reconstruction. Currently, he is working on visualization problems in postprocessing of computational fluid dynamics (CFD) simulations with CD-adapco in Austin, Texas.

Christoph Garth received the PhD (Dr rer nat) degree in computer science from the University of Kaiserslautern in 2007 and spent his postdoctoral time as a researcher with the Institute for Data Analysis and Visualization at the University of California, Davis. He is currently an assistant professor of computer science at the University of Kaiserslautern. His research interests include scientific visualization, analysis of vector and tensor fields, topological methods, query-driven visualization, and parallel/scalable algorithms for visualization. He is a member of the IEEE.

Kenneth I. Joy received the BA and MA degrees in mathematics from UCLA in 1968 and 1972, respectively, and the PhD degree from the University of Colorado, Boulder, in 1976. He is a professor in the Department of Computer Science and the director of the Institute for Data Analysis and Visualization at University of California, Davis (UC Davis). He came to UC Davis in 1980 in the Department of Mathematics and was a founding member of the Computer Science Department in 1983. He is a faculty computer scientist at Lawrence Berkeley National Laboratory and is a participating guest researcher at Lawrence Livermore National Laboratory. His areas of interest lie in the fields of visualization, geometric modeling, and computer graphics, where he leads research efforts in multiresolution representations of large scale data sets, visualization of multidimensional data, applications and visualization algorithms to imaging problems, and simplification of data sets resulting from terascale simulations. He is a member of the Association for Computing Machinery (ACM) and the IEEE.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/publications/dlib.

Authorized licensed use limited to: Huawei Technologies Co Ltd. Downloaded on March 22,2026 at 07:54:57 UTC from IEEE Xplore. Restrictions apply.